

[Sign In](#)[Free Startup Course](#)

Achieving Flow in a Startup



ASH MAURYA
BOOTSTRAPPING
FOCUS

Eliminating waste is the fundamental principle of lean thinking.

As defined by Womak/Jones in their book “Lean Thinking” (a must-read):

Waste is any human activity which absorbs resources but creates no value.

Of all resources, there is no resource more valuable than time. Time is more valuable than money. While money can fluctuate up or down, time only moves in one direction—down.

Another way in which the value of time shows up for lean startups is through Boyd’s

Subscribe to the Newsletter

Join thousands of founders for battle-tested recipes, strategies, and how-tos for achieving product/market fit systematically.

Send
Me
Weekly
Startup
Tips →

SHARE THIS ARTICLE ON:



Law which states that:

Speed of iterations beats quality of iteration.

Colonel John Boyd, a military strategist, and Air Force fighter, found this by studying an anomaly in dogfights where an inferior aircraft (F-86) consistently beat a superior aircraft (MiG-15) because it was able to iterate faster thanks to a hydraulic versus manual flight stick. Eric Ries applies Boyd's Law to lean startups by highlighting the importance of maximizing cycle time through the build/measure/learn loop.

Startups that succeed are those that manage to iterate enough times before running out of resources. Time between these iterations is fundamental.

The Conflicting Pull for Time

Time, like any resource, has multiple pulls. In following customer development, there is a basic pull for activities outside the building versus inside the building. Steve Blank asserts that all the answers lie outside the building and advocates the creation of a cross-functional customer development team **that must include the founders**. What about work that needs to get done inside the building? Who is going to implement the solutions to problems uncovered outside the building?

Eric Ries' answer is to create two teams that feed each other: a problem team and a solution team. The first team focuses on customer development, while the second team focuses on product development. However, if you are a founder, you must be on both teams, wherein lies the fundamental scheduling tug-of-war.

The problem is further exacerbated if you are a technical founder (like me) because time is utilized very differently when going from product development to customer development. Paul Graham wrote an excellent essay on the two types of schedules: manager's schedule and maker's schedule. Managers typically organize their day into 1-hour blocks and spend each hour dealing with a different task. Makers, like programmers and writers, need to organize their day into longer blocks of uninterrupted time. The cost of context switching is low (and expected) in a Manager's schedule. It is high (and a productivity killer) in a Maker's schedule.

Activities outside the building (customer interviews, usability testing, customer support) tend to be on a Manager's schedule, while activities inside the building (design, coding) are usually on a maker's schedule. Trying to find an equilibrium point between these two pulls is more art than science, but there is a fundamental concept that must be present to maximize productivity—**flow**.

There are two different definitions of what I mean by flow, and both apply here.

The first comes from psychologist Mihály Csíkszentmihályi who defines flow as a mental state of operation when you are at your best. When you are in flow, you are so totally immersed in an activity that nothing else matters. You lose your self-consciousness and sense of time.

Activities that flow typically have the following attributes:

- Have a clear objective
- Need your full concentration
- Lack of interruptions and distractions
- Clear and immediate feedback on progress towards the objective
- A sense of challenge

While flow can't be triggered at will, you can arrange activities to allow for flow which coincidentally is also the second definition of flow. From Lean Thinking:

When we start thinking about the ways to line up essential steps to get a job done to achieve a steady continuous flow with no wasted energy, batches, or queues, it changes everything including how we collaborate and the tools we devise to get the job done.

What follows are specific work hacks I use to allow for flow:

Creating Daily Flow

I generally group my daily activities into three categories: Planned maker activities, planned manager activities, and unplanned maker/manager activities.

Work Hack 1: Establish uninterrupted time blocks for maker work.

My planned maker activities are typically coding and writing tasks I've previously identified. Because these activities need an uninterrupted block of time, I schedule these very early in the morning (6 am-8 am). I usually schedule this task the night before, which is the first and only thing I do. I don't check email/Twitter or look at anything else. No one is calling at that hour, so distractions are minimal. I find two-hour blocks work best for me.

Work Hack 2: Achieve maker goals as early in the day as possible.

I've tried both staying up late and waking up early, and prefer the latter as it isn't interrupted by sleep which allows the day's activities to flow better. I also personally find that accomplishing something tangible that early in the day sets the tone for the rest of the day.



Depending on the day of the week, I might allocate more 2-hour blocks later in the morning or afternoon, but they aren't as intense as the first one and can be interrupted by something more urgent.

Work Hack 3: Schedule manager activities as late in the day as possible.

Planned manager activities, like customer meetings, are easier to schedule because they are clearly time-boxed and calendar driven. Unless there is an unworkable schedule conflict, I prefer to schedule these for the afternoon so as not to interrupt my morning flow.

Work Hack 4: Always be ready for unplanned activities, especially customer support.

Unexpected interruptions can surface from anywhere throughout the day—server issues, customer support calls, etc. You have to be prepared for the interruption, especially from customers. Both server alerts and customer calls (800 number) are routed directly to my cellphone. This is also a good place to apply a five whys process to ensure unexpected incidents don't become recurring.

Creating Weekly Flow

Aside from organizing the day for flow, I also group certain tasks and activities by day of the week.



Work Hack 5: Identify the best days for planned customer development.

For instance, Mondays and Fridays are usually bad days for initiating new customer contact as people are generally either recovering from the weekend or getting ready for it. I plan these types of customer development activities between Tuesday-Thursday.

Work Hack 6: Take advantage of customer downtime.

Since Mondays and Fridays are usually slower from a customer perspective, I use them for larger maker tasks like writing blog posts. My blog posts are usually identified on Friday, outlined roughly over the weekend, written/proofed on Monday, and published on Tuesday.

Work Hack 7: Balance face time with customers.

Not all customer development activities require face time. Beyond the initial customer discovery stage, there is a strong tendency to rely more heavily on asynchronous communication using tools like email, forums, and online usability testing. While all these tools are great for lowering real-time distractions and achieving scale, I find it important to still create opportunities for face time with existing and new customers.

Unscripted conversations are the best way to

learn about unscripted problems. I put our 800 number on all pages and encourage customers to pick up the phone versus email whenever possible.

Eliminating software waste

Building software to specifications is hard enough that when faced with a startup environment where both problems and solutions are largely unknown, it is optimal to iterate around less code than more code.

Work Hack 8: Avoid overproduction by making customers pull for features.

Customer pull is another concept from lean thinking, and it requires that no good or service be produced until a customer asks for it. At this stage, more than 80% of my effort is spent on optimizing existing features versus building new ones. The whole point of customer development is identifying an MVP that resonates with customers, and the whole point of customer validation is testing if that resonance will scale. If it doesn't, the solution is not adding features but possibly pivoting and going back to step 1—customer discovery.

Work Hack 9: Iterate around only 3–5 actionable metrics.

I've covered my conversion dashboard in detail in previous posts. Having a few actionable metrics is all I need to identify and prioritize the most critical

build/measure/learn loops to tackle. All the data for the dashboard is collected automatically and presented in a one-page report that I can view in real-time.

Work Hack 10: Build software to flow.

You might have noticed I don't have days or tasks identified for building, testing, or releasing software. That is because I follow a continuous deployment process (also popularized by Eric Ries) where software is built, tested, and packaged automatically at the end of every maker task with no effort on my part other than checking in code. One-click, and the code is released to customers.

Going back to Womak/Jones definition of Waste:

Waste is any **human activity** which absorbs resources but creates no value.

Manufacturing processes have traditionally been arranged around machine time, breaking tasks into batches and queues. Lean thinking challenges this approach and calls for arranging around human time and organizing tasks, so they flow. Releasing software is not unlike manufacturing. While it is somewhat easier to continuously deploy web-based software, with a little discipline, desktop-based software too can be built to flow. I will be detailing my continuous deployment process in a subsequent post.